

Handling Bad Data in EPC and Infrastructure Projects

Lecture Notes and Instructor Guide for a Live Google Colab Tutorial

Prepared for Bechtel Private Limited AI/ML Training

Duration: 3.5–4 hours

Contents

1 Purpose of the Session	2
2 Learning Outcomes	2
3 Recommended Session Structure	2
4 Training Storyline	3
4.1 Dataset A: Synthetic EPC Work-Package Dataset	3
4.2 Dataset B: NYC Open Infrastructure Data	3
5 Opening Motivation	4
5.1 Example 1: Wrong Units, Wrong Productivity	4
5.2 Example 2: Missing Does Not Mean Zero	4
5.3 Example 3: Duplicate Is Not Always Duplicate	4
6 Level 1: Easy - Look Before You Clean	4
6.1 Goal	4
6.2 Core Notebook Cells	4
6.3 What Participants Should Notice	5
7 Level 1 Data-Quality Report	5
7.1 Teaching Point	5
8 Level 1: Exact Duplicates and Business Duplicates	5
8.1 Exact Duplicate	5
8.2 Business Duplicate	6
9 Level 2: Moderate - Clean with EPC Logic	6
9.1 Goal	6
9.2 Cleaning Categories	6
9.3 Discipline Mapping	6
9.4 Material Availability Mapping	7
10 Cleaning Human-Formatted Costs	7
10.1 Instructor Note	7
11 Cleaning and Validating Dates	7
12 Engineering Sanity Checks	8

13 Cleaning Log and Traceability	8
14 Level 2 Domain Case: Negative Working Height	9
14.1 Naive Interpretation	9
14.2 Domain Interpretation	9
14.3 Better Feature	9
15 Level 3: Deeper - Let Model Errors Teach Us	10
15.1 Goal	10
15.2 Naive ML Workflow	10
15.3 Domain-Informed ML Workflow	10
15.4 Fair Comparison	10
15.5 One-Hot Encoding	10
15.6 Residual Diagnosis	10
16 NYC Open Data Section	11
16.1 Suggested Questions	11
16.2 Robust Loading Pattern	11
17 EPC Data-Quality Checklist	11
17.1 Structure	12
17.2 Field Meaning	12
17.3 Missingness	12
17.4 Duplicates	12
17.5 Dates	12
17.6 Numeric Validity	12
17.7 Units	13
17.8 Traceability	13
18 Suggested Slide-Level Outline	13
19 Participant Exercises	13
19.1 Exercise 1: First Inspection	13
19.2 Exercise 2: Missing Values	14
19.3 Exercise 3: Business Rules	14
19.4 Exercise 4: Model Failure Diagnosis	14
20 Common Mistakes to Warn Against	14
21 Closing Message	14

1 Purpose of the Session

This lecture-cum-notebook tutorial is designed for EPC construction professionals who need to work with project data before applying AI/ML. The session is not meant to be a generic Pandas tutorial. It is meant to teach a practical habit:

Before predictive analytics, interrogate the dataset like an engineer.

In EPC projects, AI/ML does not begin with model training. It begins with asking whether the data is meaningful, trustworthy, and decision-ready. A model trained on poorly understood data can produce confident but misleading results. In the EPC setting, this can affect schedule decisions, cost forecasts, procurement priorities, safety monitoring, quality/rework interventions, and contractor-performance reviews.

Key teaching message. Bad data is not only missing values. Bad data is any data representation, reporting convention, field definition, transformation, or storage practice that blocks reliable insight.

2 Learning Outcomes

By the end of the session, participants should be able to:

1. Identify common bad-data patterns in EPC and infrastructure records.
2. Use Python/Pandas to inspect and profile a messy dataset.
3. Distinguish exact duplicates from business duplicates.
4. Detect missingness, invalid values, inconsistent categories, and impossible dates.
5. Clean human-formatted costs, dates, categorical labels, and yes/no fields.
6. Apply engineering and project-control rules before applying statistical rules.
7. Maintain a cleaning log so that every data-cleaning decision is traceable.
8. Understand how model residuals can reveal hidden data or domain issues.
9. Prepare a dataset for downstream AI/ML only after understanding its limitations.

3 Recommended Session Structure

Time	Segment	Purpose
0:00–0:20	Why bad data matters in EPC AI/ML	Motivate using project decisions and AI risk.
0:20–0:50	Easy inspection	Rows, columns, data types, missingness, first questions.
0:50–1:25	Physical/project sanity checks	Progress, dates, costs, manhours, duplicates.

1:25–1:35	Break	Short pause.
1:35–2:15	Moderate cleaning	Categories, costs, dates, yes/no fields, cleaning log.
2:15–2:50	Domain interpretation	Negative working height: bad data or excavation signal?
2:50–3:25	Deeper ML diagnostic loop	Naive ML vs domain-informed ML on the same test set.
3:25–3:45	NYC open infrastructure data	Reality check using public infrastructure records.
3:45–4:00	Takeaways	EPC data-quality checklist and automation ideas.

4 Training Storyline

The final notebook follows an easy $\rightarrow$ moderate $\rightarrow$ deeper progression.

4.1 Dataset A: Synthetic EPC Work-Package Dataset

The synthetic dataset is self-contained and generated inside the notebook. It represents work-package-level construction data. Each row may be interpreted as:

one work package at one site during one reporting period.

The dataset intentionally includes controlled data-quality problems:

- inconsistent discipline names,
- inconsistent contractor names,
- missing values,
- duplicate-like records,
- human-formatted cost values,
- mixed date formats,
- negative or impossible manhours,
- progress values outside physical limits,
- yes/no fields with many encodings,
- free-text remarks,
- negative working height values,
- outlier productivity values.

The synthetic dataset is useful because it guarantees that participants encounter the important bad-data cases in a controlled teaching environment.

4.2 Dataset B: NYC Open Infrastructure Data

The notebook also contains a guarded section for a real NYC infrastructure dataset. The purpose is not to prove that this public dataset contains every EPC issue. Instead, it demonstrates that even official public data requires interpretation before analytics.

Useful questions for the NYC section include:

- What does one row represent?
- Which fields are project identifiers, agency fields, budget-like fields, or schedule-like fields?
- Which columns have high missingness?
- Can delay or cost overrun be defined directly?
- What additional internal EPC data would be required for a reliable ML model?

5 Opening Motivation

5.1 Example 1: Wrong Units, Wrong Productivity

A work package appears highly productive. Later, we discover that some installed quantities were recorded in feet while others were recorded in meters. A model trained on this data may learn unit inconsistency instead of productivity.

5.2 Example 2: Missing Does Not Mean Zero

A missing safety incident value may mean:

- zero incidents,
- report not submitted,
- not applicable,
- not yet entered,
- data lost during integration.

Treating all missing values as zero can create a false safety picture.

5.3 Example 3: Duplicate Is Not Always Duplicate

Two rows may not be identical, but they may refer to the same project, site, work package, and reporting week. In project controls, this is a business duplicate even when it is not an exact duplicate.

6 Level 1: Easy - Look Before You Clean

6.1 Goal

The first level is meant to make every participant comfortable with data inspection. No machine learning is introduced here. The focus is on reading the table, understanding what a row means, and asking simple questions.

Key teaching message. Do not clean immediately. First understand what the data represents.

6.2 Core Notebook Cells

```
# First look
print(df.shape)
df.head()
```

```
# Column names and types
df.info()
```

```
# Missing values
missing = df.isna().sum().sort_values(ascending=False)
missing.head(15)
```

```
# Simple numerical summary
df.describe(include="all").T
```

6.3 What Participants Should Notice

- Some columns that look numeric may be stored as text.
- Cost fields may include symbols, commas, or currency labels.
- Date fields may not be parsed as dates.
- Missing values have different meanings in different columns.
- Categorical labels may contain inconsistent capitalization or spelling.

Instructor prompts. Ask participants:

- What does one row represent?
- Which columns would you trust immediately?
- Which columns require a data dictionary?
- Which fields are engineering fields and which are project-control fields?

7 Level 1 Data-Quality Report

A compact data-quality report helps participants summarize the first inspection.

```
def data_quality_report(df):
    return pd.DataFrame({
        "column": df.columns,
        "dtype": df.dtypes.astype(str).values,
        "missing_count": df.isna().sum().values,
        "missing_pct": (df.isna().mean() * 100).round(2).values,
        "unique_values": df.nunique(dropna=True).values
    }).sort_values("missing_pct", ascending=False)

quality_report = data_quality_report(df)
quality_report.head(20)
```

7.1 Teaching Point

This report is not a final answer. It is a triage tool. It helps decide where the instructor and participants should look next.

8 Level 1: Exact Duplicates and Business Duplicates

8.1 Exact Duplicate

```
df.duplicated().sum()
```

8.2 Business Duplicate

In EPC records, a business key may be something like:

project_id + site_id + work_package_id + reporting_week

```
business_key = ["project_id", "site_id", "work_package_id", "
               reporting_week"]
df.duplicated(subset=business_key).sum()
```

Key teaching message. A row can be a project-control duplicate even when it is not an exact duplicate in the CSV file.

9 Level 2: Moderate - Clean with EPC Logic

9.1 Goal

The second level introduces cleaning, but the emphasis remains on engineering judgement. Participants should learn that some values are truly bad, while others only look bad until the domain is understood.

9.2 Cleaning Categories

EPC data often contains categorical variables entered manually by different teams.

```
def clean_text(x):
    if pd.isna(x):
        return np.nan
    x = str(x).strip().lower()
    x = re.sub(r"\s+", " ", x)
    return x

for col in ["discipline", "contractor", "material_available"]:
    df[col + "_clean"] = df[col].apply(clean_text)
```

9.3 Discipline Mapping

```
discipline_map = {
    "civil": "Civil",
    "civ": "Civil",
    "civil works": "Civil",
    "structural": "Structural",
    "structure": "Structural",
    "mech": "Mechanical",
    "mechanical": "Mechanical",
    "elec": "Electrical",
    "electrical": "Electrical",
    "piping": "Piping",
    "pipe": "Piping"
}

df["discipline_clean"] = df["discipline_clean"].map(discipline_map).
    fillna("Unknown")
```

9.4 Material Availability Mapping

```
availability_map = {
    "yes": 1, "y": 1, "available": 1, "1": 1, "true": 1,
    "no": 0, "n": 0, "not available": 0, "0": 0, "false": 0
}

df["material_available_clean"] = (
    df["material_available_clean"].map(availability_map)
)
```

Key teaching message. Cleaning categories is not merely spelling correction. It changes the meaning of groups used in dashboards and ML models.

10 Cleaning Human-Formatted Costs

Cost data may appear as numbers, strings, strings with commas, strings with currency labels, or values copied from Excel. The goal is to convert them to numeric values while preserving the raw column.

```
def clean_money(x):
    if pd.isna(x):
        return np.nan
    x = str(x)
    x = x.replace(",", "")
    x = x.replace("$", "")
    x = x.replace("USD", "")
    x = x.replace("INR", "")
    x = x.replace("Rs.", "")
    x = x.strip()
    try:
        return float(x)
    except ValueError:
        return np.nan

df["cost_planned_clean"] = df["cost_planned"].apply(clean_money)
df["cost_actual_clean"] = df["cost_actual"].apply(clean_money)
```

10.1 Instructor Note

Do not silently overwrite the original cost column. Create a new cleaned column and preserve the raw source column.

11 Cleaning and Validating Dates

Date parsing is not enough. After parsing, dates must satisfy project logic.

```
date_cols = ["planned_start", "planned_finish", "actual_start", "actual_finish"]

for col in date_cols:
    df[col + "_clean"] = pd.to_datetime(df[col], errors="coerce",
                                         dayfirst=True)
```



```
df["planned_duration_days"] = (
    df["planned_finish_clean"] - df["planned_start_clean"]
).dt.days

df["actual_duration_days"] = (
    df["actual_finish_clean"] - df["actual_start_clean"]
).dt.days

bad_dates = df[df["actual_duration_days"] < 0]
```

Key teaching message. A valid date format does not imply a valid project record. A finish date before a start date is a project-logic issue, not merely a parsing issue.

12 Engineering Sanity Checks

Some checks should be based on physical or project-control constraints.

Field	Sanity Check	Interpretation
Progress	$0 \leq \text{progress} \leq 100$	Values above 100 may indicate overrun, wrong denominator, or entry error.
Manhours	$\text{manhours} \geq 0$	Negative manhours are usually invalid.
Actual duration	$\text{finish} \geq \text{start}$	Violations need correction or escalation.
Cost	$\text{cost} \geq 0$	Negative cost may need accounting explanation.
Installed quantity	$\text{quantity} \geq 0$	Negative quantity may be invalid unless it represents reversal/rework.

```
issues = {}
issues["progress_gt_100"] = (df["actual_progress_clean"] > 100).sum()
issues["negative_manhours"] = (df["manhours_spent_clean"] < 0).sum()
issues["negative_actual_duration"] = (df["actual_duration_days"] < 0).sum()
issues
```

13 Cleaning Log and Traceability

A cleaning log records what changed, where, and why. This is important in EPC because cleaned data may influence contractual, schedule, cost, or safety decisions.

```
cleaning_log = []

def log_cleaning_step(step, column, action, reason):
    cleaning_log.append({
        "step": step,
        "column": column,
```

```

        "action": action,
        "reason": reason
    })

log_cleaning_step(
    step="Standardized discipline names",
    column="discipline",
    action="Mapped CIV, civil works, structure, mech, elec to standard
          labels",
    reason="Same discipline was recorded using multiple labels"
)

log_cleaning_step(
    step="Parsed cost fields",
    column="cost_planned, cost_actual",
    action="Removed symbols and converted to numeric values",
    reason="ML and numerical summaries require numeric cost fields"
)

cleaning_log_df = pd.DataFrame(cleaning_log)
cleaning_log_df

```

Key teaching message. The raw data should be preserved. Cleaning should create new columns, flags, and logs rather than silently destroying the original record.

14 Level 2 Domain Case: Negative Working Height

This is one of the most important examples in the tutorial.

14.1 Naive Interpretation

A negative working height may initially look like bad data:

```
working_height_m = -2.5
```

A naive cleaning rule may delete all negative values.

14.2 Domain Interpretation

In construction, negative height may represent below-grade work, basement excavation, trenching, or underground utility activity. Therefore, deleting negative height values may remove an entire operational regime.

Key teaching message. Some data is not wrong. It is a domain signal that has been encoded in a way that a naive analyst may misunderstand.

14.3 Better Feature

If scaffold length or work access demand depends on the magnitude of height, one may use:

```

df["abs_working_height_m"] = df["working_height_m"].abs()
df["below_grade_flag"] = (df["working_height_m"] < 0).astype(int)

```

This preserves both magnitude and below-grade meaning.

15 Level 3: Deeper - Let Model Errors Teach Us

15.1 Goal

The third level uses a small ML model not as the main objective, but as a diagnostic tool. The idea is:

A model that fails systematically may be telling us something about the data.

15.2 Naive ML Workflow

In the first workflow, we apply naive cleaning, such as deleting negative working heights. We then train a simple model to predict scaffold length or construction support quantity.

15.3 Domain-Informed ML Workflow

In the second workflow, we preserve negative working heights and add domain-informed features such as absolute height and below-grade indicator.

15.4 Fair Comparison

The improved notebook evaluates both models on the same test indices. This avoids a misleading comparison where Model 1 and Model 2 are tested on different data.

```
train_idx, test_idx = train_test_split(
    model_df.index,
    test_size=0.25,
    random_state=42
)
```

15.5 One-Hot Encoding

For linear models, categorical variables such as discipline should be one-hot encoded, not label encoded.

```
model_df = pd.get_dummies(
    df[features + [target]],
    columns=["discipline_clean"],
    drop_first=True
)
```

Key teaching message. Label encoding can falsely imply order: Civil = 0, Electrical = 1, Mechanical = 2. One-hot encoding avoids this artificial ordering.

15.6 Residual Diagnosis

After training, residuals should be plotted and interpreted.

```
results["error"] = results["actual"] - results["predicted"]
results["abs_error"] = results["error"].abs()

plt.scatter(results["actual"], results["error"])
plt.axhline(0, linestyle="--")
plt.xlabel("Actual value")
```

```
plt.ylabel("Prediction error")
plt.title("Residual plot")
plt.show()
```

Instructor prompts. Ask participants:

- Where does the model fail most strongly?
- Are errors random or systematic?
- Do errors concentrate for below-grade work?
- Does the failure reveal a data issue or a missing domain feature?

16 NYC Open Data Section

The NYC section is included as a real-data reality check. It should be run after the synthetic EPC exercise, not before it.

16.1 Suggested Questions

- What does one row mean in this public dataset?
- Which columns are complete and which are sparse?
- Which fields appear to represent budgets, agencies, locations, or project status?
- Can we define delay from the available columns?
- Can we define cost overrun from the available columns?
- What additional EPC internal data would be needed for serious AI/ML?

16.2 Robust Loading Pattern

```
try:
    nyc = pd.read_csv(nyc_url)
    print("NYC dataset loaded:", nyc.shape)
except Exception as e:
    nyc = None
    print("Could not load NYC dataset.")
    print(e)
```

Each subsequent NYC cell should check whether the dataset loaded successfully.

```
if nyc is not None:
    display(nyc.head())
else:
    print("NYC dataset not loaded. Continue with synthetic EPC section.")
```

17 EPC Data-Quality Checklist

Use this checklist before building any AI/ML model on EPC data.

17.1 Structure

- Can the file be read reliably?
- Is the delimiter correct?
- Is each row one work package, one activity, one contractor, one day, or one week?
- Are there hidden Excel-formatting or merged-cell problems?

17.2 Field Meaning

- What does each column mean?
- What is the unit?
- What is the source system?
- Who owns the field?
- Is the value manually entered or system-generated?

17.3 Missingness

- Which columns have missing values?
- Does missing mean unknown, not applicable, ongoing, not reported, or zero?
- Is missingness concentrated in specific contractors, sites, or disciplines?

17.4 Duplicates

- Are there exact duplicates?
- Are there business duplicates?
- What is the business key?
- Which record is authoritative?

17.5 Dates

- Are dates parseable?
- Are multiple formats mixed?
- Are finish dates before start dates?
- Are planned and actual dates defined consistently?

17.6 Numeric Validity

- Are manhours non-negative?
- Is progress between 0 and 100?
- Are costs parseable as numbers?
- Are extremely high or low values explainable?

17.7 Units

- Are quantities reported in mixed units?
- Are all quantities comparable within a discipline?
- Are unit conversions documented?

17.8 Traceability

- Is raw data preserved?
- Is every cleaning step logged?
- Can the cleaned dataset be reproduced?
- Are corrected values distinguished from original values?

18 Suggested Slide-Level Outline

Slide	Content
1	Handling Bad Data in EPC and Infrastructure Projects.
2	Why this matters: bad data creates bad AI; bad AI creates bad project decisions.
3	Sources of EPC data: P6, ERP, site reports, procurement trackers, safety logs, Excel, emails, PDFs, IoT.
4	Bad data is more than missing values.
5	Common EPC bad-data examples.
6	Synthetic EPC dataset: one row = work package $\times$ site $\times$ reporting period.
7	First principle: look before cleaning.
8	Data quality report: dtype, missingness, unique values.
9	Exact duplicates vs business duplicates.
10	Categorical cleaning: discipline, contractor, material availability.
11	Dates: parseability versus project logic.
12	Numeric cleaning: progress, costs, quantities, manhours.
13	Missingness: zero, unknown, ongoing, not applicable.
14	Negative height: bad data or below-grade work?
15	Cleaning log and traceability.
16	Model 1: naive cleaning.
17	Residual diagnosis: what did the model fail to learn?
18	Model 2: domain-informed features.
19	NYC open data reality check.
20	Final takeaway: trustworthy decisions are the real goal.

19 Participant Exercises

19.1 Exercise 1: First Inspection

Ask participants to inspect the first 10 rows and answer:

1. What does one row represent?
2. Which columns are likely manually entered?

3. Which columns are likely system-generated?
4. Which columns would you not trust without a data dictionary?

19.2 Exercise 2: Missing Values

For each missing field, classify it as one of:

unknown, zero, not applicable, ongoing, delayed reporting, source-system failure.

19.3 Exercise 3: Business Rules

Write three EPC-specific validation rules. Examples:

- Actual finish cannot be earlier than actual start.
- Progress should normally lie between 0 and 100.
- A completed work package should have an actual finish date.

19.4 Exercise 4: Model Failure Diagnosis

After looking at residuals, ask:

1. Which records have the largest error?
2. Are they concentrated in one discipline or regime?
3. Is the issue a model limitation, data issue, or missing domain feature?

20 Common Mistakes to Warn Against

1. Dropping rows with missing values without understanding why they are missing.
2. Treating missing safety incidents as zero without confirming reporting practice.
3. Label encoding nominal categories for linear models.
4. Removing outliers that correspond to real operational events.
5. Overwriting raw columns during cleaning.
6. Training ML models before defining the target variable carefully.
7. Treating official or public data as automatically clean.
8. Reporting accuracy without checking whether the model learned reporting artifacts.

21 Closing Message

In EPC projects, data cleaning is not clerical work. It is engineering reasoning applied to project records. The aim is not to make a CSV look neat. The aim is to create a trustworthy pipeline from site reporting to project decisions.

Clean data is not the goal. Trustworthy decisions are the goal.